

MOCK INTERVIEW PREP GUIDE

Java Full Stack Development — Questions with Expected Answers

Cognizant Digital Nurture 5.0 — Deep Skilling Handbook (Java FSE / React Track)

Student Preparation Copy — Final-Year CSE Batch | Department of Computer Engineering & Applications

How to Use This Guide

This guide covers the same 12 topic areas and questions used in your mock interview, with a model answer under each question.

- Treat these as a starting point, not a script to memorize word-for-word. Interviewers can tell the difference between understanding and recitation.
- For every answer, try to also prepare a short real example from a project you've built (the microservices project, the JPA/CRUD project, or the design pattern examples covered in class).
- Difficulty tags: Green = Easy (know this cold), Amber = Medium (be able to explain with an example), Red = Hard (be ready to reason through trade-offs, not just define).
- Practice saying answers out loud, not just reading them — interview pressure changes how well you can recall things.

1. Core Java & OOP Concepts

Q1. Explain the four pillars of OOP with a real example from a project you've built. [Easy]

A: The four pillars are Encapsulation (bundling data and methods together and restricting direct access via private fields and public getters/setters), Inheritance (a class acquiring properties/behavior of a parent class using extends), Polymorphism (the same method behaving differently based on the object — method overriding/overloading), and Abstraction (exposing only essential details through abstract classes or interfaces while hiding implementation). Example: in a food-delivery app, a Payment interface with UPIPayment and CardPayment implementations demonstrates abstraction and polymorphism, while a Order class with private fields and public getters demonstrates encapsulation.

Q2. What is the difference between method overloading and method overriding? [Easy]

A: Overloading happens within the same class — multiple methods share a name but differ in parameter list (number, type, or order); it is resolved at compile time (static polymorphism). Overriding happens across a parent-child class relationship — a subclass provides a specific implementation of a method already defined in the parent with the same signature; it is resolved at runtime based on the actual object type (dynamic polymorphism), and requires the method to not be static, final, or private in the parent.

Q3. Why is Java called 'platform independent'? What role does the JVM play? [Easy]

A: Java source code is compiled into an intermediate form called bytecode (.class files), not directly into machine code. The JVM (Java Virtual Machine), which is platform-specific, interprets/JIT-compiles this bytecode into native instructions for whichever OS it runs on. So the same .class file runs unchanged on Windows, Linux, or macOS as long as a compatible JVM is installed — 'write once, run anywhere'. The JDK includes the compiler and tools, the JRE includes the JVM plus libraries needed to run applications.

Q4. What is the difference between == and .equals() for objects and Strings? [Easy]

A: == compares references (memory addresses) for objects — it checks whether two variables point to the same object instance. .equals() compares logical/content equality and its behavior depends on whether the

class has overridden it (Object's default .equals() also just does reference comparison). For Strings, literals created without new are interned in the String pool, so two identical literals may share a reference and == may return true; but new String("abc") creates a new object on the heap, so == would return false even though .equals() returns true.

Q5. Explain the difference between abstract classes and interfaces. When would you choose one over the other? [Medium]

A: An abstract class can have both abstract and concrete methods, constructors, instance fields, and any access modifier; a class can extend only one abstract class (single inheritance). An interface (pre-Java 8) could only declare method signatures and constants; since Java 8 it can also have default and static methods with bodies, but still no instance state, and a class can implement multiple interfaces. Choose an abstract class when there's shared state or common implementation among closely related classes; choose an interface to define a contract/capability that unrelated classes can implement, or when multiple inheritance of type is needed.

Q6. What is immutability? How would you design an immutable class? [Medium]

A: An immutable object's state cannot change after construction. To design one: declare the class as final (prevent subclassing that could break invariants), make all fields private and final, don't provide setters, initialize all fields via the constructor, and if a field is a mutable object (like a Date or a List), return defensive copies from getters and store defensive copies in the constructor rather than the original reference. String and Java's wrapper classes (Integer, Long, etc.) are classic examples.

Q7. Explain how the Garbage Collector works at a high level. What can cause a memory leak in Java despite GC? [Medium]

A: The JVM heap is divided into generations (Young/Eden+Survivor and Old/Tenured). Most objects die young, so minor GC frequently scans the Young generation and promotes surviving objects to Old gen; Old gen is collected less often via major/full GC. The GC uses reachability analysis from GC roots (mark-and-sweep style) — anything unreachable is eligible for collection. Despite this, memory leaks can still occur if objects remain reachable unintentionally: static collections that keep growing, unclosed resources (streams, connections), listeners/callbacks that are registered but never removed, or ThreadLocal values not cleared.

Q8. What is the difference between StringBuilder, StringBuffer, and String? When would you use each? [Medium]

A: String is immutable — every concatenation creates a new object, which is wasteful in loops. StringBuilder is mutable and not thread-safe, giving the best performance for single-threaded string building (e.g. building a large string in a loop). StringBuffer is mutable and thread-safe (synchronized methods), used when multiple threads modify the same string buffer concurrently, at some performance cost compared to StringBuilder.

Q9. Explain multithreading basics: how do you create a thread, and what is the difference between Runnable and Thread? [Medium]

A: A thread can be created either by extending the Thread class and overriding run(), or by implementing the Runnable interface and passing an instance to a Thread constructor, then calling start() (never call run() directly — that just runs it on the current thread). Implementing Runnable is generally preferred because Java doesn't support multiple inheritance — implementing an interface keeps the class free to extend something else, and it separates the task (what to run) from the thread mechanics (how to run it). In modern code, ExecutorService and thread pools are preferred over manually creating threads.

Q10. What are static and instance initializer blocks, and when does each run? [Hard]

A: A static initializer block (static { ... }) runs exactly once, when the class is first loaded by the JVM, before any object is created — used for static field setup. An instance initializer block ({ ... } without static) runs every time an object is created, right before the constructor body executes, after the superclass constructor call. Execution order for object creation: static blocks (once, on class load) → instance blocks → constructor body, and this happens for every new object except the static part which only runs the first time.

2. Collections Framework & Java 8 Features

Q1. Compare ArrayList and LinkedList in terms of performance and use cases. [Easy]

A: ArrayList is backed by a dynamic array, giving $O(1)$ random access (get/set by index) but $O(n)$ insertion/deletion in the middle since elements must shift. LinkedList is a doubly linked list, giving $O(1)$ insertion/deletion once you have a reference to the node (e.g. at the head/tail) but $O(n)$ random access since it must traverse from an end. Use ArrayList when you mostly read/iterate; use LinkedList when you frequently insert/remove from the ends (like a queue/dequeue) — though in practice ArrayDeque often outperforms LinkedList for that too.

Q2. What is the difference between HashMap, LinkedHashMap, and TreeMap? [Easy]

A: HashMap offers no ordering guarantee and $O(1)$ average time for get/put, backed by a hash table. LinkedHashMap maintains insertion order (or access order if configured) by additionally keeping a doubly linked list, with a slight overhead over HashMap. TreeMap keeps keys sorted according to their natural ordering or a supplied Comparator, backed by a red-black tree, giving $O(\log n)$ operations — used when sorted iteration matters.

Q3. How does HashMap handle collisions internally? [Medium]

A: A key's hashCode() is used (with additional bit-spreading) to compute a bucket index in the internal array. When two keys hash to the same bucket (a collision), Java 8+ stores them as a linked list of entries in that bucket by default, and checks equals() to find the right key on lookup. If a single bucket's chain grows beyond a threshold (8 entries, with the table large enough), it's converted (treeified) into a red-black tree for that bucket to keep worst-case lookup at $O(\log n)$ instead of $O(n)$.

Q4. What is the difference between Comparable and Comparator? [Medium]

A: Comparable is implemented by the class itself, defining a single 'natural ordering' via compareTo() (e.g. Employee implements Comparable<Employee>). Comparator is a separate class/lambda implementing compare(a, b), defined externally to the class, allowing multiple different sort orders without modifying the original class — useful when you need to sort the same objects different ways in different contexts (by name in one place, by salary in another).

Q5. Explain lambda expressions and functional interfaces with an example. [Medium]

A: A functional interface is an interface with exactly one abstract method (marked optionally with @FunctionalInterface), such as Runnable, Comparator, or custom ones. A lambda expression provides a concise inline implementation of that single method without an anonymous class, e.g. (a, b) -> a.getSalary() - b.getSalary() as a Comparator<Employee>. Common built-in functional interfaces include Function<T,R> (takes T returns R), Predicate<T> (takes T returns boolean), Supplier<T> (takes nothing returns T), and Consumer<T> (takes T returns nothing).

Q6. Walk through a Stream API pipeline that filters, transforms, and collects a list — e.g. get names of employees earning above a threshold. [Medium]

A: `employees.stream().filter(e -> e.getSalary() > threshold).map(Employee::getName).collect(Collectors.toList())`. `stream()` converts the collection into a Stream, `filter()` is a lazy intermediate operation keeping only elements matching the predicate, `map()` transforms each Employee into its name (a String), and `collect(Collectors.toList())` is the terminal operation that triggers execution and gathers results into a List. Intermediate operations aren't executed until a terminal operation is called (lazy evaluation), which lets the JVM optimize the pipeline.

Q7. What are default and static methods in interfaces? Why were they introduced in Java 8? [Medium]

A: A default method provides a method body in an interface using the default keyword, which implementing classes inherit automatically unless they choose to override it. A static method belongs to the interface itself and is called via `InterfaceName.method()`, not through an instance. They were introduced so existing interfaces (like List) could gain new methods (like `stream()` or `forEach()`) without breaking every class that already implements them — otherwise adding a method to an interface would force all implementers to add an implementation immediately.

Q8. What is the Optional class and what problem does it solve? [Hard]

A: `Optional<T>` is a container object that may or may not hold a non-null value, used as a return type to explicitly signal that a value might be absent, forcing the caller to handle that case (via `isPresent()`, `ifPresent()`, `orElse()`, `orElseThrow()`, `map()`) instead of accidentally causing a `NullPointerException`. It's best used as a method return type for values that may legitimately be absent, not as a field type or method parameter, and not overused for every nullable value — misuse can add unnecessary complexity.

3. Exception Handling

Q1. What is the difference between checked and unchecked exceptions? Give one example of each. [Easy]

A: Checked exceptions are checked by the compiler at compile time — a method must either handle them with try-catch or declare them with throws; they represent recoverable conditions like `IOException` or `SQLException`. Unchecked exceptions (subclasses of `RuntimeException`) are not checked at compile time and typically represent programming errors, like `NullPointerException`, `ArithmeticException` (divide by zero), or `ArrayIndexOutOfBoundsException`.

Q2. Explain try-with-resources. Why is it preferred over a manual finally block for closing resources? [Easy]

A: try-with-resources (`try (Resource r = ...) { ... }`) automatically closes any resource that implements `AutoCloseable` when the try block exits, whether normally or via an exception — the `close()` call happens implicitly, in reverse order for multiple resources. This is preferred over manually closing in a finally block because it's less boilerplate, avoids forgetting to close a resource, and correctly suppresses/reports exceptions from `close()` alongside the original exception rather than silently swallowing one of them.

Q3. Can you have a try block without a catch block? Explain the possible combinations. [Medium]

A: Yes — a try block must be followed by at least one of catch or finally, but not necessarily both. Valid combinations: try-catch, try-finally (no catch, just guaranteed cleanup, exception still propagates), try-catch-finally, and try-with-resources which can itself omit catch/finally since the resource is auto-closed (though you can still add catch/finally around it).

Q4. How would you design a custom exception hierarchy for a banking or food-delivery application? [Medium]

A: Create a base application exception, e.g. `BankingException` extends `RuntimeException` (or `Exception` if you want it checked), then specific subclasses like `InsufficientFundsException`, `AccountNotFoundException`, `InvalidTransactionException` extending it. This lets calling code catch broadly (catch `(BankingException e)`) or specifically as needed, and a global handler can map each subclass to an appropriate HTTP status. Whether to make it checked or unchecked depends on whether callers can reasonably be expected to recover — most modern APIs favor unchecked exceptions for business rule violations to avoid forcing try-catch everywhere.

Q5. What is exception chaining and why is it useful? [Medium]

A: Exception chaining means wrapping a lower-level exception as the 'cause' of a new, higher-level exception, typically via a constructor like `new ServiceException("message", originalException)`. It's useful because it preserves the original stack trace and root cause (accessible via `getCause()`) while letting each layer of the application throw exceptions meaningful to its own abstraction level, without losing the detailed technical reason for debugging.

Q6. In a Spring Boot REST API, how would you implement centralized exception handling? [Hard]

A: Use a class annotated with `@ControllerAdvice` (or `@RestControllerAdvice`) containing methods annotated with `@ExceptionHandler(SpecificException.class)` for each exception type you want to handle, returning a consistent error response body (e.g. containing timestamp, status, message, path) along with an appropriate HTTP status via `ResponseEntity`. This avoids duplicating try-catch logic in every controller and ensures the API returns uniform, predictable error responses (e.g. mapping `ResourceNotFoundException` to 404, validation errors to 400).

4. Spring Core & Spring Boot

Q1. What is Dependency Injection and why does Spring use it? Name the types of DI Spring supports. [Easy]

A: Dependency Injection is a design pattern where an object's dependencies are provided ('injected') by an external container rather than the object creating them itself, achieving loose coupling and making code easier to test (dependencies can be mocked). Spring supports constructor injection (dependencies passed via constructor — recommended, since it allows fields to be final and makes required dependencies explicit), setter injection (via setter methods, useful for optional dependencies), and field injection (via `@Autowired` directly on a field — convenient but discouraged since it hides dependencies and complicates testing).

Q2. What is the Spring IoC container and what is a Bean? [Easy]

A: The IoC (Inversion of Control) container, represented by `ApplicationContext`, is responsible for instantiating, configuring, wiring together, and managing the lifecycle of objects in a Spring application. A Bean is simply an object that is created, configured, and managed by this container instead of being manually instantiated with 'new' by application code — beans are typically defined via annotations (`@Component`, `@Service`, etc.) or Java/XML configuration.

Q3. What are the different Spring bean scopes? Explain singleton vs prototype. [Medium]

A: Common scopes: singleton (default — exactly one shared instance per Spring container, reused for every injection), prototype (a new instance is created every time the bean is requested/injected), and web-aware scopes like request (one instance per HTTP request) and session (one instance per HTTP session). Singleton is appropriate for stateless services; prototype is used when a bean holds mutable, request-specific state that shouldn't be shared.

Q4. What is the difference between @Component, @Service, @Repository, and @Controller? [Medium]

A: All four are specializations ('stereotypes') of `@Component`, so Spring's component scanning detects and registers all of them as beans identically. The distinction is semantic/documentational — `@Service` marks business/service-layer logic, `@Controller` marks a web layer component handling requests (`@RestController` combines `@Controller` + `@ResponseBody`), and `@Repository` marks the data-access layer, with the added technical benefit that Spring wraps repository beans with exception translation, converting database-specific exceptions into Spring's unified `DataAccessException` hierarchy.

Q5. Explain the Spring Bean lifecycle — from creation to destruction. [Medium]

A: The container instantiates the bean (calls its constructor), then injects dependencies (setter/field injection after constructor injection), then calls any `Aware` interface callbacks (e.g. `ApplicationContextAware`), then invokes any `@PostConstruct`-annotated method or `afterPropertiesSet()` (if `InitializingBean` is implemented) for custom initialization logic, after which the bean is ready for use. On container shutdown, it calls any `@PreDestroy` method or `destroy()` (if `DisposableBean` is implemented) for cleanup, then the bean is discarded.

Q6. What auto-configuration does Spring Boot provide, and how does `@SpringBootApplication` tie things together? [Medium]

A: `@SpringBootApplication` is a composite annotation combining `@Configuration` (marks the class as a source of bean definitions), `@EnableAutoConfiguration` (tells Spring Boot to automatically configure beans based on what's on the classpath — e.g. auto-configuring a `DataSource` if a JDBC driver is present), and `@ComponentScan` (scans the current package and sub-packages for components). Auto-configuration classes use conditional annotations like `@ConditionalOnClass` and `@ConditionalOnMissingBean` so they only activate when appropriate and don't override beans you've defined yourself.

Q7. How would you externalize configuration (e.g. DB URL, ports) across dev/test/prod environments in Spring Boot? [Medium]

A: Use separate property files per environment, e.g. `application-dev.yml`, `application-prod.yml`, activated via `spring.profiles.active=dev` (set as an environment variable or JVM argument), with shared defaults in `application.yml`. Classes/beans can be restricted to a profile using `@Profile("dev")`. For type-safe grouped configuration, use `@ConfigurationProperties` classes bound to a prefix in the YAML. Sensitive values (passwords, keys) are best supplied via environment variables or a secrets manager rather than committed to the property files.

Q8. What is Spring AOP and where have you used it, or where would it be useful? [Hard]

A: Aspect-Oriented Programming lets you modularize cross-cutting concerns — logic that would otherwise be duplicated across many classes, like logging, transaction management, security checks, or performance monitoring — into a separate 'aspect' that's woven in around specific method executions (a 'pointcut'). Using `@Aspect` with advice types like `@Before`, `@After`, `@Around`, you can, for example, log the execution time of every method in the service layer without adding logging code to each method individually. Spring's own `@Transactional` is itself implemented using AOP proxies.

Q9. How does Spring Boot's embedded server (Tomcat) differ from a traditional WAR deployment model? [Hard]

A: Traditionally, you'd package a Java web app as a WAR file and deploy it into an external, separately-installed servlet container (like a standalone Tomcat or JBoss), which you had to manage independently. Spring Boot instead embeds Tomcat (or Jetty/Undertow) directly inside an executable JAR, so the application is self-contained — you run it with `'java -jar app.jar'` and it starts its own server, with no separate container

installation needed. This simplifies deployment, especially in containerized (Docker) and cloud environments, at the cost of the app owning its own server configuration rather than sharing one.

Q10. What is circular dependency between beans, and how would you resolve it? *[Hard]*

A: A circular dependency occurs when Bean A depends on Bean B, and Bean B (directly or transitively) depends back on Bean A, so neither can be fully constructed first. With constructor injection, Spring fails fast at startup with a `BeanCurrentlyInCreationException` since it can't resolve the cycle. With setter/field injection, Spring can sometimes resolve it by injecting a partially-initialized bean, or you can use `@Lazy` on one of the dependencies to defer its resolution until first use. The better long-term fix is usually to refactor the design — extract the shared logic into a third bean, or reconsider whether the two beans are too tightly coupled.

5. REST API Design

Q1. Explain REST principles. What makes an API 'RESTful'? *[Easy]*

A: REST (Representational State Transfer) is an architectural style built around resources identified by URIs, manipulated via standard HTTP methods (GET, POST, PUT, DELETE, PATCH). Key principles: statelessness (each request contains all information needed; the server holds no client session state between requests), a uniform interface (consistent resource-based URLs and standard verbs/status codes), representations (resources are exchanged as JSON/XML rather than exposing internal implementation), and, ideally, HATEOAS (responses include links to related actions), though most real-world 'RESTful' APIs skip full HATEOAS compliance.

Q2. What is the difference between PUT and PATCH? Between POST and PUT? *[Easy]*

A: PUT replaces the entire resource with the payload provided and is idempotent (calling it multiple times with the same data has the same effect as calling it once). PATCH applies a partial update — only the fields provided are changed. POST is used to create a new resource (or trigger a non-idempotent action) and is not idempotent — calling it twice typically creates two resources, whereas PUT to a specific known URI is idempotent by design.

Q3. How do you handle request validation in a Spring Boot REST controller? *[Medium]*

A: Annotate the request body/DTO fields with Bean Validation constraints like `@NotNull`, `@NotBlank`, `@Size`, `@Min/@Max`, `@Email`, then annotate the controller method parameter with `@Valid` (or `@Validated` for group validation). If validation fails, Spring throws a `MethodArgumentNotValidException`, which you catch in a `@ControllerAdvice/@ExceptionHandler` to return a 400 Bad Request with a clear, structured list of field errors rather than a generic stack trace.

Q4. How would you version a REST API, and what are the trade-offs of different approaches? *[Medium]*

A: Common approaches: URI versioning (`/api/v1/orders` — simple and visible, but clutters the URI and can be seen as violating the idea that a URI identifies a resource, not a version of it), header versioning (a custom header or Accept header carries the version — keeps URIs clean but is less discoverable/harder to test casually), and query-parameter versioning (`?version=1` — easy to add but easy to forget/misuse). Most teams pick URI versioning for its simplicity and visibility despite the theoretical objections.

Q5. Explain the difference between `@RequestParam`, `@PathVariable`, and `@RequestBody`. *[Medium]*

A: `@RequestParam` extracts values from the query string (e.g. `?page=2` in `/orders?page=2`). `@PathVariable` extracts values embedded in the URI path template (e.g. the `{id}` in `/orders/{id}`). `@RequestBody` deserializes the entire HTTP request body (typically JSON) into a Java object, used for POST/PUT/PATCH payloads.

Q6. How would you design pagination and sorting for a GET endpoint returning a large list? [Medium]

A: Accept page and size query parameters (and optionally sort, e.g. sort=name,asc), map them to a Spring Data Pageable object, and pass that into a repository method returning Page<T> or Slice<T>. The response should include metadata like content, totalElements, totalPages, and current page number, so the client can build pagination controls without a separate count query.

Q7. What HTTP status codes would you return for: resource created, resource not found, validation failure, and unauthorized access? [Medium]

A: 201 Created for a successfully created resource (typically with a Location header pointing to the new resource), 404 Not Found when the requested resource doesn't exist, 400 Bad Request for validation/malformed-input failures, 401 Unauthorized when the caller isn't authenticated at all (no or invalid credentials), and 403 Forbidden when the caller is authenticated but lacks permission for that specific action.

Q8. How do you document and test a REST API before handing it to a frontend team? [Hard]

A: Use springdoc-openapi (or springfox) to auto-generate interactive Swagger/OpenAPI documentation directly from the controller annotations, so the frontend team can see all endpoints, request/response schemas, and try calls in the browser. Complement this with a Postman collection covering happy-path and edge-case scenarios, which can also be shared and version-controlled. Writing integration tests (e.g. with MockMvc or RestAssured) before handoff also acts as executable documentation and catches regressions.

6. Spring Data JPA / Hibernate

Q1. What is the difference between JPA and Hibernate? [Easy]

A: JPA (Java Persistence API) is just a specification — a set of interfaces and annotations (like @Entity, @Id, EntityManager) defining how Java objects should be mapped to relational database tables, without specifying implementation. Hibernate is a concrete implementation (provider) of that specification, and is the most widely used one; Spring Data JPA sits on top of JPA and provides repository abstractions, with Hibernate typically as the underlying provider.

Q2. Explain the difference between JpaRepository, CrudRepository, and PagingAndSortingRepository. [Easy]

A: They form a hierarchy: CrudRepository provides basic CRUD operations (save, findById, findAll, deleteById, etc.). PagingAndSortingRepository extends it, adding paginated and sorted retrieval (findAll(Pageable), findAll(Sort)). JpaRepository extends PagingAndSortingRepository and adds JPA-specific operations like batch deletion, flushing changes immediately (saveAndFlush), and returning List instead of Iterable for some methods. In practice, most Spring Data JPA repositories simply extend JpaRepository to get everything.

Q3. What is the N+1 select problem, and how would you fix it? [Medium]

A: It happens when you fetch a list of N parent entities with one query, and then, because an association is lazily loaded, accessing that association on each parent triggers a separate query per entity — resulting in 1 + N queries instead of a single efficient one. Fixes: use JOIN FETCH in a JPQL query to eagerly fetch the association in the same query, use @EntityGraph to specify which associations to load eagerly for a particular query, or configure batch fetching (hibernate.default_batch_fetch_size) to fetch associations in batches instead of one at a time.

Q4. Explain the difference between FetchType.LAZY and FetchType.EAGER, and when you'd choose each. [Medium]

A: LAZY means the associated data is only loaded from the database when it's actually accessed (a proxy is returned initially), which is good for performance when you don't always need the association. EAGER means the associated data is loaded immediately along with the owning entity, which is simpler but can hurt performance by pulling in data you don't need, or contribute to N+1 problems. By default, @OneToMany and @ManyToMany are LAZY, while @ManyToOne and @OneToOne are EAGER — it's generally recommended to make most associations LAZY explicitly and fetch what's needed via JOIN FETCH or entity graphs when required.

Q5. How do @OneToMany, @ManyToOne, and @ManyToMany mappings work? Give an example from a project. [Medium]

A: @ManyToOne is the 'owning side' typically — e.g. many Orders belong to one Customer, so Order has a Customer field annotated @ManyToOne with a foreign key column. @OneToMany is the inverse side — Customer would have a List<Order> annotated @OneToMany(mappedBy = "customer") pointing back, where mappedBy indicates Order owns the relationship/foreign key. @ManyToMany, e.g. Students and Courses, requires a join table (often auto-created, or explicitly defined with @JoinTable) since neither side can hold a single foreign key to represent the many-to-many link.

Q6. What is the persistence context / first-level cache in Hibernate? [Medium]

A: The persistence context (managed by the EntityManager/Session) is a first-level cache scoped to a single transaction/session — once an entity is loaded or saved within that context, subsequent requests for the same entity (by ID) within the same session return the cached managed instance instead of hitting the database again. It also enables 'dirty checking': Hibernate tracks changes to managed entities and automatically generates UPDATE statements at flush/commit time without needing an explicit save() call for modifications.

Q7. How does @Transactional work in Spring, and what happens if an exception is thrown inside a transactional method? [Hard]

A: @Transactional is implemented via a proxy (JDK dynamic proxy or CGLIB) that wraps the annotated method: it begins a transaction before the method runs and commits it after, provided the method completes normally. By default, Spring rolls back the transaction automatically only for unchecked exceptions (RuntimeException and its subclasses) or Errors — checked exceptions do NOT trigger rollback by default, so you'd need rollbackFor = Exception.class to include them. Propagation settings (REQUIRED, REQUIRES_NEW, etc.) control how nested @Transactional calls interact with an existing transaction.

Q8. How would you handle database schema evolution across environments (e.g. adding a column without breaking production)? [Hard]

A: Use a migration tool like Flyway or Liquibase, where each schema change is captured as a versioned, ordered migration script checked into source control and applied automatically on application startup (or via a pipeline step) consistently across dev/QA/prod. This is preferred over hibernate.ddl-auto=update in production, which can make unpredictable or destructive changes automatically and offers no audit trail or rollback path — ddl-auto is generally only safe for local development or throwaway environments.

7. Microservices Architecture

Q1. What is a microservices architecture, and what problems does it solve compared to a monolith? [Easy]

A: A microservices architecture splits an application into a set of small, independently deployable services, each owning its own data and business capability, communicating over the network (typically REST or messaging). Compared to a monolith, it allows teams to develop, deploy, and scale services independently, use different technologies per service if needed, and limits the blast radius of a failure or a bad deployment to one service rather than the whole application — at the cost of added operational complexity (network calls, distributed data consistency, more infrastructure to manage).

Q2. What is service discovery, and why is it needed? Have you used Eureka or a similar tool? [Medium]

A: In a microservices system, service instances can scale up/down and their network locations (IP/port) change dynamically, so hardcoding URLs isn't practical. Service discovery solves this: each service registers itself with a discovery server (like Netflix Eureka) on startup, and other services query the discovery server to find current, healthy instances of a service by name rather than a fixed address. Eureka has a server component (the registry) and clients (each microservice registers itself and can look up others) — combined with a client-side load balancer, this lets calls be routed to any available instance.

Q3. What is an API Gateway, and what responsibilities does it typically take on? [Medium]

A: An API Gateway is a single entry point that sits in front of all microservices, routing incoming client requests to the appropriate backend service. It commonly handles cross-cutting concerns centrally: request routing/load balancing, authentication/authorization, rate limiting, request/response transformation, and sometimes aggregating responses from multiple services — this keeps individual microservices simpler and clients from needing to know about internal service topology.

Q4. How do microservices communicate with each other? Compare synchronous (Feign/REST) and asynchronous (messaging) approaches. [Medium]

A: Synchronous communication (REST calls, often via a declarative client like OpenFeign) is simple to reason about and gives an immediate response, but couples the caller to the callee's availability — if the downstream service is slow or down, the caller is blocked or fails too. Asynchronous communication (via a message broker like Kafka or RabbitMQ) decouples services in time — the publisher doesn't wait for the consumer to process the event — improving resilience and scalability, at the cost of added complexity (eventual consistency, message ordering, harder debugging).

Q5. What is the circuit breaker pattern, and why is it important in a microservices system? [Medium]

A: A circuit breaker wraps calls to a potentially failing downstream service and monitors failure rates; if failures exceed a threshold, the circuit 'opens' and further calls fail fast (or return a fallback) without even attempting the network call, for a cooldown period — preventing the caller from repeatedly hammering an already-struggling service and avoiding resource exhaustion (threads/connections tied up waiting) that could cascade failures across the system. After the cooldown, it goes 'half-open' to test if the dependency has recovered. Resilience4j is the modern standard library for this in Spring Boot (Hystrix is now in maintenance mode).

Q6. How would you handle a transaction that spans multiple microservices, e.g. placing an order that debits an account and creates a loan record? [Hard]

A: Traditional distributed two-phase-commit transactions don't scale well and aren't practical across independently-owned microservice databases. The common solution is the Saga pattern: the overall business transaction is broken into a sequence of local transactions, each in one service, with either choreography (each service publishes an event that triggers the next step, fully decentralized) or orchestration (a central saga orchestrator explicitly tells each service what to do next). If a later step fails,

compensating transactions undo the effects of the earlier completed steps (e.g. refund the debited amount), achieving eventual consistency rather than strict atomicity.

Q7. How do you handle configuration management across many microservices? [Hard]

A: Use a centralized configuration server, such as Spring Cloud Config Server, which stores configuration (often backed by a Git repository) for all services in one place. Each microservice fetches its configuration from the config server on startup (and optionally refreshes it at runtime via Spring Cloud Bus), allowing environment-specific overrides and centralized changes without redeploying every service individually, and keeping secrets/config out of the codebase.

Q8. What challenges have you faced (or would expect) when debugging an issue that spans multiple services? [Hard]

A: The main challenge is that a single user request may pass through several services, each with its own logs, making it hard to reconstruct the full picture from scattered log files. The standard approach is distributed tracing (e.g. Spring Cloud Sleuth generating a trace ID/span ID propagated across service calls, visualized in a tool like Zipkin) combined with a correlation ID attached to every log line for a given request, and centralized log aggregation (like the ELK stack) so all services' logs can be searched together by that correlation ID.

8. Design Patterns

Q1. Explain the Singleton pattern. What problems can arise with it in a multi-threaded environment? [Easy]

A: Singleton ensures a class has exactly one instance and provides a single global access point to it, typically via a private constructor and a static getInstance() method. In a multi-threaded environment, naive 'lazy initialization' (checking if the instance is null and creating it if so) can cause a race condition where two threads both see null and create two separate instances. Fixes include synchronizing the getInstance() method (simple but adds overhead on every call), double-checked locking with a volatile field (checks null, synchronizes, checks null again before creating), or using an eagerly-initialized static field, or an enum singleton (which is inherently thread-safe and serialization-safe).

Q2. Explain the Factory Method pattern with an example. How is it different from Abstract Factory? [Medium]

A: Factory Method defines a method (often in a base class or interface) that subclasses override to decide which concrete class to instantiate, so client code depends on an abstraction rather than a concrete constructor — e.g. a NotificationFactory.createNotification(type) that returns an EmailNotification or SMSNotification depending on input. Abstract Factory is one level higher: it creates families of related objects (multiple factory methods bundled in one interface), e.g. a UIFactory that produces a matching Button, Checkbox, and Menu for either a 'Dark' or 'Light' theme, ensuring the produced objects are compatible with each other.

Q3. Where have you seen or used the Builder pattern? Why is it useful for objects with many optional fields? [Medium]

A: The Builder pattern separates the construction of a complex object from its representation, letting you set only the fields you need step by step and then call build() to get the final immutable object — avoiding 'telescoping constructors' (many overloaded constructors for different combinations of optional parameters) and making object creation more readable at the call site (e.g. Order.builder().withCustomer(c).withItems(items).build()). In real projects, Lombok's @Builder annotation generates this boilerplate automatically for POJOs with several optional fields.

Q4. Explain the Strategy pattern and give a scenario where it would replace a large if-else/switch block.

[Medium]

A: Strategy encapsulates a family of interchangeable algorithms/behaviors behind a common interface, letting the algorithm vary independently from the client that uses it — instead of a single class containing a large if-else or switch statement choosing behavior based on a type flag, each behavior becomes its own class implementing the strategy interface, and the caller is configured with (or looks up) the appropriate strategy instance. Example: a discount calculation with different rules per customer tier (Regular, Premium, VIP) — instead of if (tier == PREMIUM) {...} else if (...), each tier gets a DiscountStrategy implementation, and the checkout code simply calls strategy.calculate(order).

Q5. What is the Observer pattern, and where is it used in Spring or Java itself? [Medium]

A: Observer defines a one-to-many dependency where a subject notifies a list of registered observers automatically whenever its state changes, without the subject needing to know the concrete type of its observers — a publisher/subscriber relationship. Spring uses this via its ApplicationEvent and ApplicationListener mechanism (or the simpler @EventListener annotation): a component publishes an event (e.g. OrderPlacedEvent) via ApplicationEventPublisher, and any number of listeners can react to it (send a confirmation email, update inventory) without the publisher being coupled to those listeners.

Q6. How does the Dependency Injection pattern relate to the broader principle of 'programming to an interface, not an implementation'? [Hard]

A: DI works best when classes depend on abstractions (interfaces) rather than concrete implementations — a service depends on a PaymentGateway interface, not directly on a StripePaymentGateway class. The container then decides, at configuration time, which concrete implementation to inject. This keeps the dependent class decoupled from implementation details, makes it trivial to swap implementations (e.g. for testing, inject a mock PaymentGateway) without changing the dependent class's code, and aligns with the Open/Closed Principle — the class is open to different behaviors via different injected implementations, but closed to modification.

9. Security (Spring Security & Auth)

Q1. What is the difference between authentication and authorization? [Easy]

A: Authentication answers 'who are you?' — verifying a user's identity, typically via credentials like a username/password or a token. Authorization answers 'what are you allowed to do?' — determining whether an already-authenticated user has permission to access a specific resource or perform a specific action, typically based on roles or permissions.

Q2. How does Spring Security's filter chain work at a high level? [Medium]

A: Spring Security inserts a chain of servlet filters in front of the application's controllers. Each incoming request passes through this chain, where different filters handle different concerns in order — e.g. one filter extracts and validates a JWT or session cookie, populating the SecurityContext with the authenticated user if valid; another filter checks whether the authenticated (or anonymous) user is authorized to access the requested URL based on configured rules. Only if the request passes all relevant filters does it reach the actual controller method.

Q3. What is JWT, and why is it commonly used for securing REST APIs? [Medium]

A: A JSON Web Token is a compact, self-contained token consisting of a header, a payload (claims like user ID, roles, expiry), and a signature, Base64-encoded and concatenated. It's popular for REST APIs because the server doesn't need to store session state — the token itself carries the identity/claims and its signature

(verified using a secret or public key) proves it hasn't been tampered with, making it well-suited to stateless, horizontally-scaled services where any server instance can validate the token without checking a shared session store.

Q4. What is the difference between session-based authentication and token-based authentication?

[Medium]

A: Session-based authentication stores session state on the server (often in memory or a shared store like Redis) after login, and the client holds a session ID (usually in a cookie) that the server looks up on every request — this requires sticky sessions or a shared session store when scaling across multiple servers. Token-based authentication (like JWT) is stateless — the server verifies the token's signature on each request without storing anything server-side, which scales more easily horizontally but makes immediate token revocation (e.g. on logout) harder since the server isn't tracking active sessions.

Q5. What is CSRF, and why is it usually disabled for stateless REST APIs but not for browser-session-based apps? [Hard]

A: Cross-Site Request Forgery tricks a logged-in user's browser into unknowingly submitting a request to a site where they're authenticated (exploiting the browser automatically attaching cookies to requests), potentially performing unwanted actions on their behalf. This risk is tied to cookie-based session authentication, where the browser sends credentials automatically. Stateless REST APIs using token-based auth (where the client must explicitly attach a bearer token in a header, not something the browser sends automatically) are much less exposed to this specific attack, which is why CSRF protection is often disabled for such APIs — though other protections (like proper CORS configuration) are still needed.

Q6. How would you secure sensitive configuration like database passwords and API keys in a Spring Boot application? [Hard]

A: Never hardcode secrets in application.yml/properties or commit them to source control. Instead, supply them via environment variables or command-line arguments at deploy time, use a centralized, encrypted configuration server (Spring Cloud Config with encryption support), or a dedicated secrets manager (like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault) that the application fetches secrets from at startup, keeping them out of the codebase and version control entirely and allowing them to be rotated independently of a deployment.

10. SQL & Database

Q1. What is the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN? [Easy]

A: INNER JOIN returns only rows where there's a match in both tables based on the join condition. LEFT JOIN returns all rows from the left table plus matched rows from the right table, filling in NULLs for right-table columns where there's no match. RIGHT JOIN is the mirror image — all rows from the right table plus matched rows from the left, with NULLs where there's no match on the left.

Q2. What is normalization, and can you explain up to 3NF with a quick example? [Medium]

A: Normalization organizes a database schema to reduce data redundancy and avoid update/insert/delete anomalies, typically by decomposing tables based on functional dependencies. 1NF requires atomic column values (no repeating groups/lists in a cell). 2NF (building on 1NF) requires that every non-key column depends on the whole primary key, not just part of it (relevant for composite keys). 3NF (building on 2NF) requires that non-key columns depend only on the primary key, not on other non-key columns — e.g. if an Orders table stores CustomerCity derived from CustomerId, that's a transitive dependency that should be moved to a separate Customers table.

Q3. What is an index, and how does it affect read vs write performance? [Medium]

A: An index is a separate data structure (commonly a B-tree) built on one or more columns that allows the database to look up rows much faster than scanning the whole table, similar to an index at the back of a book. It significantly speeds up SELECT queries (and JOIN/WHERE/ORDER BY on indexed columns), but every INSERT, UPDATE, or DELETE also has to update the index structures, adding write overhead — so indexes should be added deliberately on columns that are frequently queried/filtered on, not on every column.

Q4. Write a query to find the second-highest salary in an Employee table. [Medium]

A: A common approach: `SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee)`; this finds the highest salary that is still less than the overall maximum. Alternatively, with window functions: `SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rn FROM Employee) t WHERE rn = 2` — DENSE_RANK handles ties (duplicate salaries) more gracefully than a simple LIMIT/OFFSET approach.

Q5. What is the difference between a primary key and a unique key? [Easy]

A: A table can have only one primary key, which uniquely identifies each row and cannot contain NULL values. A table can have multiple unique keys, which also enforce uniqueness across the column(s) but, depending on the database, may allow one NULL value (since NULL is not considered equal to another NULL in most engines).

Q6. How would you diagnose a slow-running query in production? [Hard]

A: Start by running EXPLAIN (or EXPLAIN ANALYZE) on the query to see the execution plan — check whether it's doing a full table scan instead of using an index, whether joins are ordered efficiently, and how many rows are being examined versus returned. Common culprits: a missing index on a filtered/joined column, an N+1 query pattern from the application layer generating many small queries instead of one efficient one, outdated table statistics misleading the query planner, or connection pool exhaustion causing queries to queue rather than actually being slow themselves.

11. Cloud Deployment & DevOps Basics

Q1. What is the difference between a Docker image and a Docker container? [Easy]

A: A Docker image is an immutable, read-only template/blueprint containing the application code, dependencies, and configuration needed to run it, built from a Dockerfile. A container is a running (or stopped) instance of that image — a lightweight, isolated process with its own filesystem layer on top of the image, created when you run 'docker run'. You can start multiple containers from the same image.

Q2. Why would you containerize a Spring Boot microservice before deploying it? [Medium]

A: Containerizing packages the application together with its exact runtime environment (JVM version, dependencies, OS-level libraries), so it behaves identically in dev, QA, and production — eliminating 'it works on my machine' issues. It also provides process isolation between services running on the same host, makes horizontal scaling straightforward (spin up more container instances), and is the standard unit of deployment for orchestration platforms like Kubernetes.

Q3. What role does a CI/CD pipeline play in delivering a Spring Boot application? [Medium]

A: A CI/CD pipeline automates the stages between committing code and having it running in production: Continuous Integration typically builds the project and runs automated tests on every commit/PR, catching regressions early and giving fast feedback. Continuous Delivery/Deployment then automates packaging (e.g.

building a Docker image), and deploying it to staging or production environments, reducing manual, error-prone release steps and enabling more frequent, reliable releases.

Q4. At a high level, what does Kubernetes give you that plain Docker doesn't? [Medium]

A: Docker runs and manages individual containers on a single host. Kubernetes is an orchestration layer that manages containers across a cluster of many machines: it automatically restarts failed containers (self-healing), scales the number of running instances up or down based on load, handles service discovery and load balancing between pods, performs rolling deployments/rollbacks with zero downtime, and schedules workloads across available nodes efficiently — things you'd otherwise have to script and manage manually with plain Docker.

Q5. How would you approach externalizing environment-specific configuration for a cloud deployment (dev/QA/prod)? [Hard]

A: Within the application, use Spring profiles (application-{profile}.yml) so environment-specific values are cleanly separated from code. In a Kubernetes deployment, non-sensitive configuration is typically supplied via ConfigMaps and sensitive values via Secrets, both injected into the container as environment variables or mounted files at deploy time — so the same container image is promoted unchanged across environments, with only the externally-supplied configuration differing. For larger systems, a centralized config server (Spring Cloud Config) can also manage this across many services from one place.

12. Behavioral / HR Round

Q1. Walk me through a project where you had to make a design trade-off. What did you choose and why? [Medium]

A: There's no single 'correct' answer here — the interviewer wants to see structured reasoning. A strong answer names the specific trade-off (e.g. 'I chose eager loading over lazy loading for this association because the API always needed the related data, and avoiding N+1 queries mattered more than the extra memory'), explains what alternatives were considered, and honestly acknowledges the downside of the choice made rather than presenting it as having no cost.

Q2. Describe a bug that was difficult to track down. How did you eventually find and fix it? [Medium]

A: A good answer walks through the debugging process concretely: how the bug was first noticed, what hypotheses were tested and ruled out, which tools were used (logs, breakpoints, a profiler, checking a stack trace), what the actual root cause turned out to be, and — ideally — what was done afterward to prevent a similar issue (a test added, a code review checklist item, better logging).

Q3. How do you approach learning a new framework or technology under a deadline? [Easy]

A: A strong answer shows a structured approach: starting with official documentation or a well-regarded quick-start guide rather than random tutorials, building a small proof-of-concept to validate understanding before applying it to the real task, and being willing to ask a teammate or mentor rather than getting stuck for hours alone — prioritizing progress on the actual deadline over exhaustively learning every feature upfront.

Q4. Tell me about a time you disagreed with a teammate's technical approach. How was it resolved? [Medium]

A: The interviewer is evaluating communication and collaboration, not just who was 'right'. A good answer explains the disagreement calmly and factually, shows that the candidate tried to understand the teammate's reasoning first, presented their own view with evidence (not just opinion), and describes how it

was actually resolved — through discussion, a compromise, deferring to data/testing, or accepting the teammate's approach gracefully if it turned out to be better.